

DSA MID LAB-EXAM

CS - 23

Time: 3 hour

Total : 50

Q1: Character Count in a Binary Image

Objective:

Implement an algorithm to count the total number of distinct characters in a binary image represented in a 2D array. Each character is represented by a cluster of '1's (white pixels), surrounded by '0's (black pixels).

Input:

A binary image represented as a 2D matrix of size m x n, where each element is either '0' (black) or '1' (white).

Output:

An integer representing the total number of distinct characters (clusters of '0's) in the binary image.

Example Input:

```
[  
  [1,0,1,1,0,1,1,1,0,0,0],  
  [1,0,0,0,0,1,1,1,1,0,1],  
  [1,0,0,0,0,1,1,1,1,0,1],  
  [1,0,1,1,0,1,1,1,0,0,0],  
  [1,1,1,1,1,1,1,1,1,1,1],  
  [1,1,1,0,1,1,1,1,0,0,0],  
  [1,1,0,1,0,1,1,1,1,0,1],  
  [1,0,0,0,0,0,1,1,1,0,1],  
  [0,1,1,1,1,1,0,1,0,0,0],  
]
```

Example Output:

4 (in the example input, the characters consist of four distinct clusters of '0's, "HI AI").

Description:

A character is defined as a contiguous group of '0's that can be connected either horizontally or vertically, diagonally. The algorithm should traverse the binary image to identify and count these characters without counting any '0's in the same character more than once.

```
int countCharacters(vector<vector<int>>image);
```

Q2: Exponentially Growing Linked List

Design and implement a custom LinkedList class where the size of each node's value array grows exponentially, and accessing elements by index must be achieved in constant time, $O(1)$. Specifically, the first node should hold 1 value, the second node 2 values, the third node 4 values, and so on, with each subsequent node doubling the number of values it stores.

Requirements:

1. Exponential Growth Structure: The first node holds 1 value, the second node holds 2 values, the third node holds 4 values, and so forth. This exponential growth should continue as values are added to the list.

2. Constant-Time Access: Provide an efficient mechanism to access values by index in $O(1)$ time, despite the varying node sizes.

3. Insertion: Implement an insertion method that appends new values to the list while maintaining the exponential growth of node sizes. The insertion should handle the dynamic allocation of new nodes as needed.

4. Deletion: Implement a deletion method that allows for removing values by index, updating both the data structure and auxiliary tracking information to maintain $O(1)$ access.

Example: If you insert 10 values, they would be distributed across the nodes as follows:

First node: [value1] (1 element)

Second node: [value2, value3] (2 elements)

Third node: [value4, value5, value6, value7] (4 elements)

Fourth node: [value8, value9, value10] (remaining 3 elements)

Implement following methods:

```
void insert(int index,int value);  
void delete(int index);  
int get(int index); // To access index 5, the system should determine it lies in the  
third node, at position 2, in  $O(1)$  time.
```